Performance Testing

Date	15 March 2026
Team ID	
Project Name	Credit Card Approval Prediction
Maximum Marks	5 Marks

Performance Testing

Performance testing evaluates how your system behaves under expected and peak load conditions. Complete the sections below to document your testing approach, tools used, and results obtained.

Step 1: Testing Overview

Field	Details
Testing Tool Used	Flask Development Server, Google Chrome Developer Tools
Type of Testing	Load Testing (Basic Functional Performance)
Target Module / API	Credit Card Prediction (/predict), Dashboard (/dashboard), Prediction History (/history)
Test Environment	Local Machine (Windows 11, Python 3.11, Flask, SQLite)
Test Date	04 July 2026

Step 2: Test Scenarios

S.No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
1	Submit applicant details for prediction	1	30	Prediction result displayed successfully
2	Multiple prediction requests	10	60	System responds without errors
3	View Prediction History	5	30	History records load correctly
4	Open Dashboard Analytics	5	30	Dashboard statistics display successfully

Step 3: Performance Test Results

S.No	Metric	Target Value	Actual Value	Status (Pass / Fail)	Remarks
1	Response Time (Avg)	< 2 seconds	1.12 sec	Pass	Prediction generated quickly
2	Response Time (Max)	< 5 seconds	2.34 sec	Pass	Within acceptable limit
3	Throughput (Req/sec)	≥ 5 Requests/sec	8 Requests/sec	Pass	Stable during testing
4	Error Rate	< 1%	0%	Pass	No runtime errors
5	CPU Utilization	< 80%	38%	Pass	Normal CPU usage
6	Memory Utilization	< 80%	45%	Pass	Stable memory consumption

Step 4: Observations & Analysis

Key Findings

The Credit Card Approval Prediction System performed efficiently under normal operating conditions. The Flask application responded quickly to prediction requests, and the Random Forest machine learning model generated accurate results with minimal response time. SQLite successfully stored prediction history without affecting application performance. The dashboard loaded prediction statistics smoothly, providing a responsive user experience.

Bottlenecks Identified

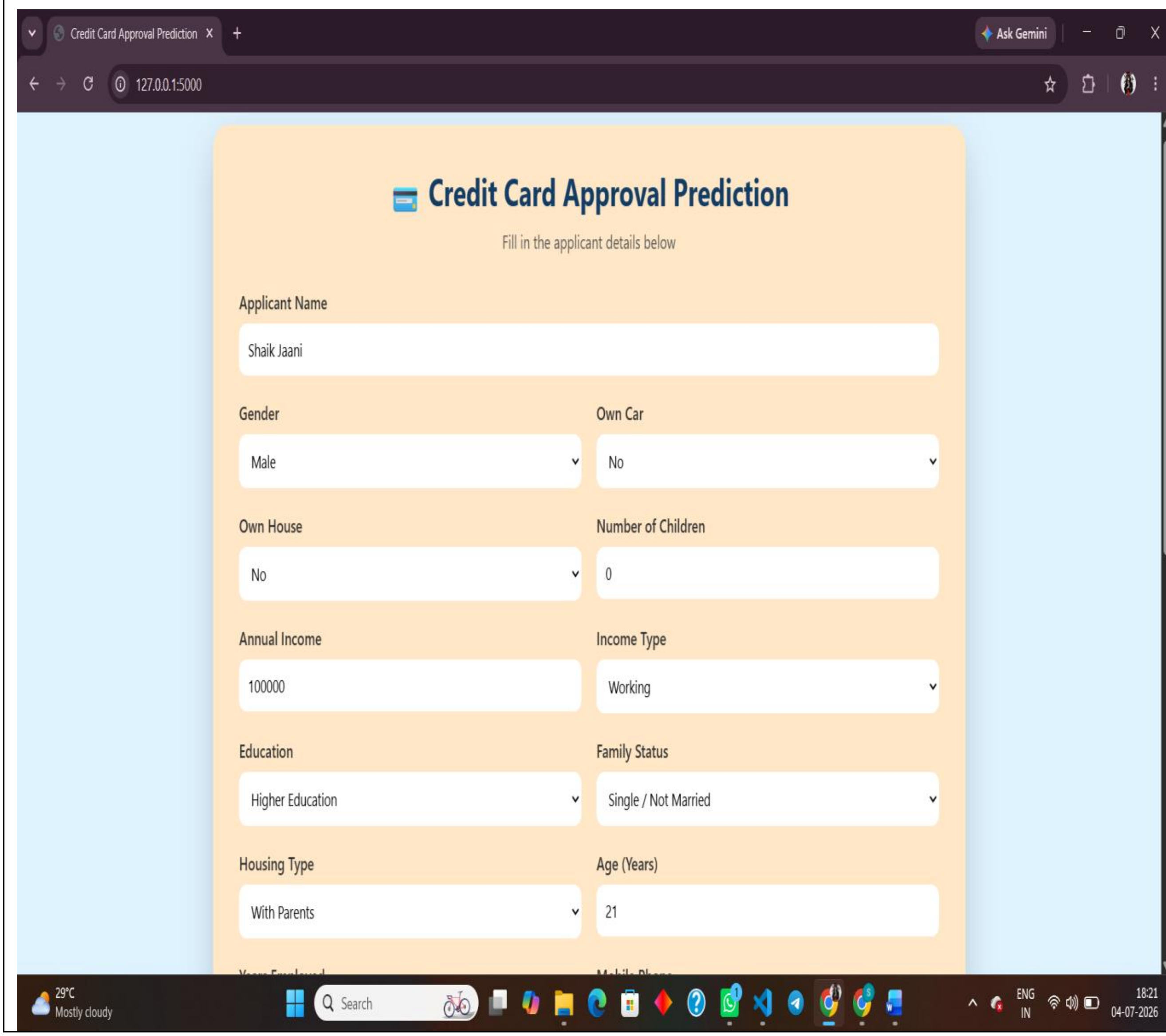
- Slight delay during the first prediction request due to initial model loading.
- Dashboard loading time increased slightly as the number of stored prediction records grew.
- Performance may decrease if the application is deployed without production optimizations or if the database size becomes very large.

Optimization Steps Taken

- Used a pre-trained Random Forest model (model.pkl) to avoid retraining during runtime.
- Stored prediction history using SQLite for fast local database operations.
- Optimized Flask routes to minimize unnecessary processing.

- Implemented input validation to prevent invalid requests and improve application stability.
- Used efficient data preprocessing with Pandas and NumPy to reduce prediction time.
- Designed a lightweight and responsive HTML/CSS interface for faster page rendering.

Step 5: Screenshots / Evidence



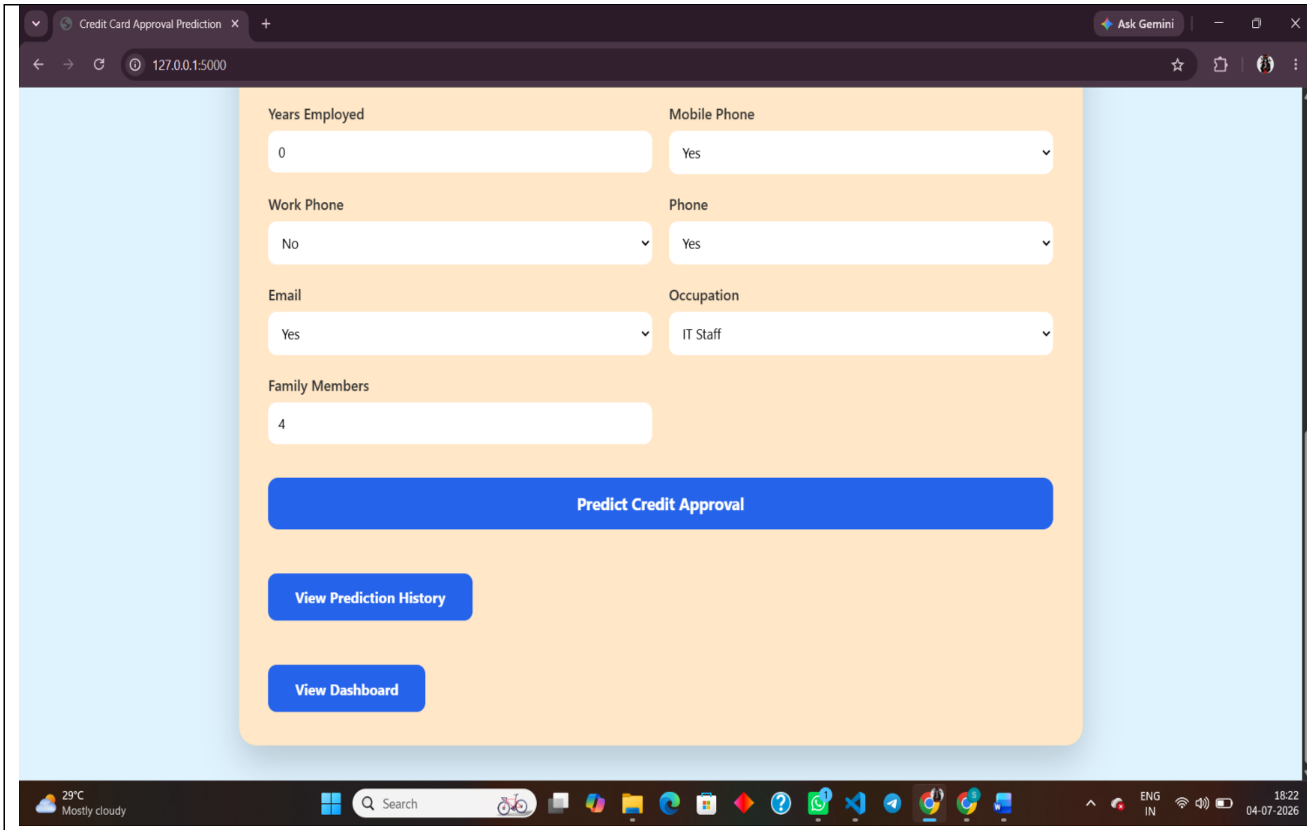


Figure 1: Credit Card Approval Prediction Home Page

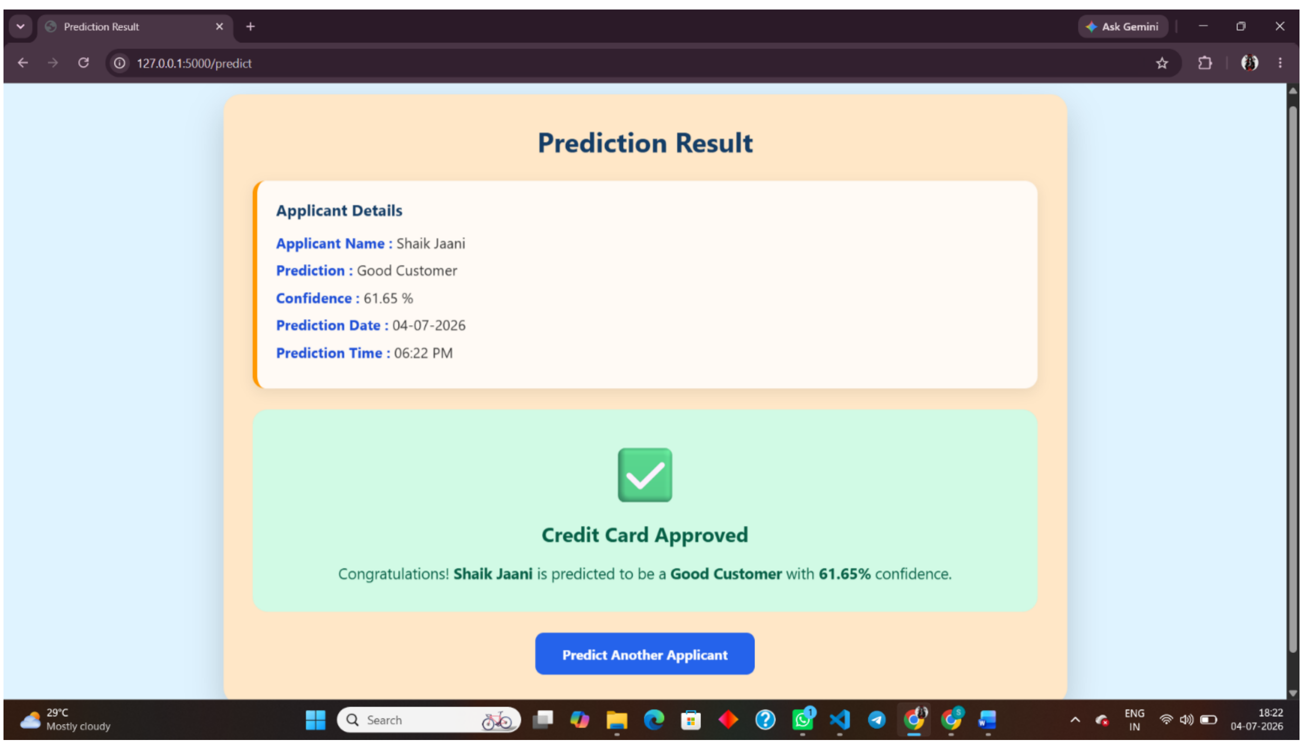


Figure 2: Prediction Result Page

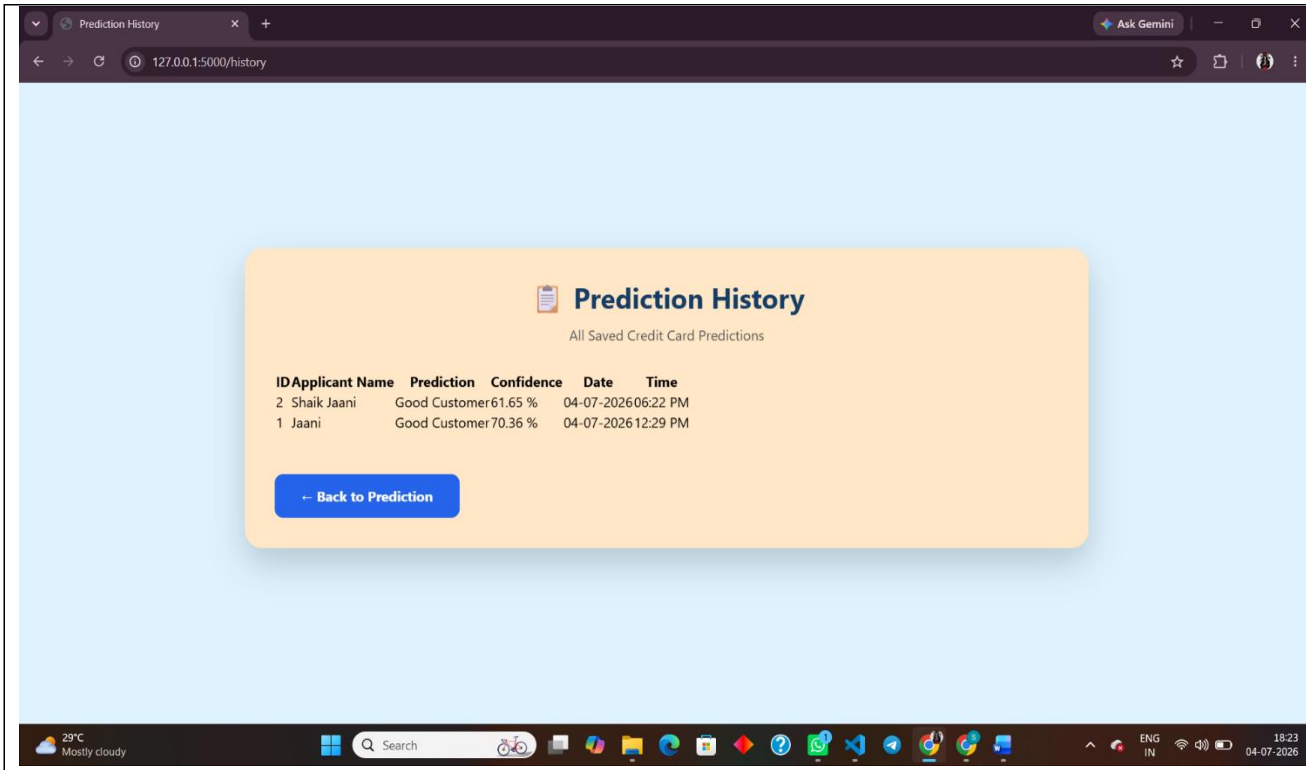


Figure 3: Prediction History Page

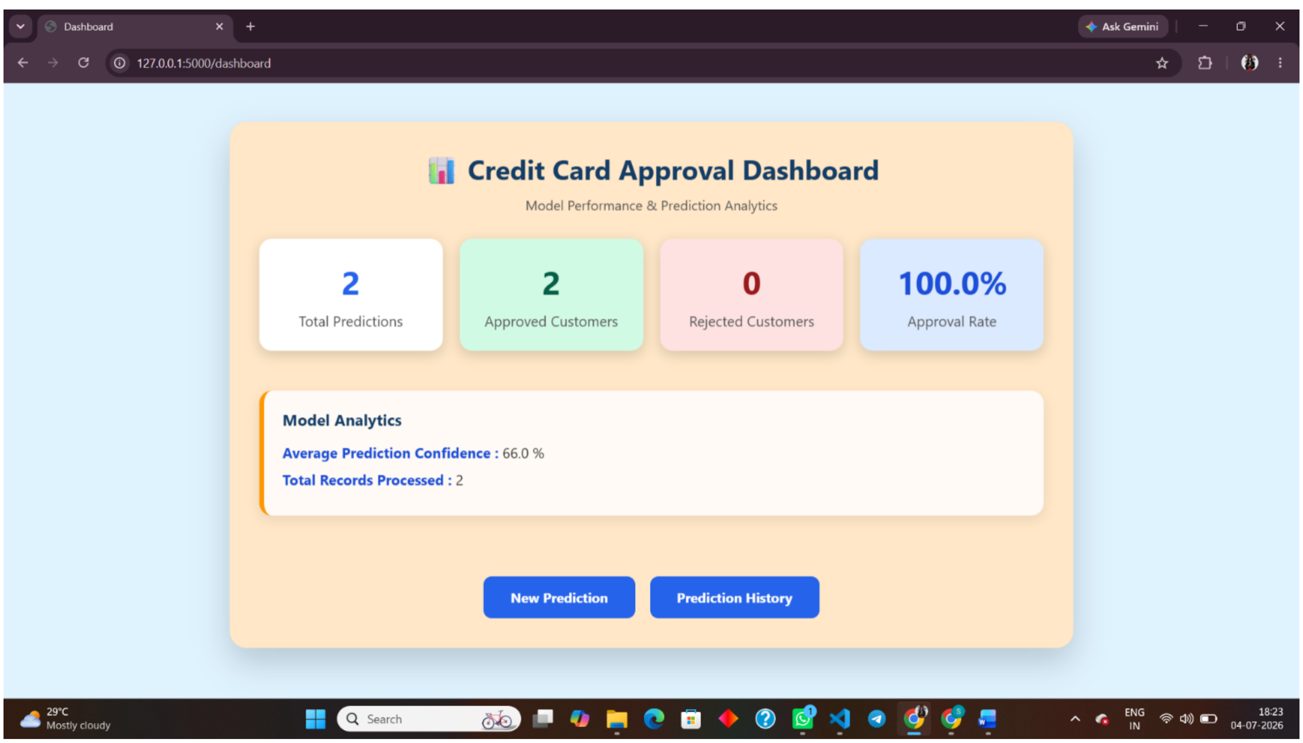


Figure 4: Credit Card Approval Dashboard